

## **7. LAS RUTINAS. PROCEDIMIENTOS Y FUNCIONES.**

Cuando se presenta la necesidad de ejecutar un mismo grupo de sentencias en distintos lugares del programa, conviene disponer de algún recurso para no tener que escribirlas repetidamente, lo cual resulta tedioso y presenta otros inconvenientes.

La solución que se suele ofrecer consiste en agrupar cada uno de esos conjuntos de sentencias destacándolos y asignándoles un nombre, constituyendo así las llamadas RUTINAS o subprogramas. Luego, basta con invocar su nombre en cualquier sentencia para conseguir que se ejecute todo el conjunto.

En PASCAL, se pueden manejar dos tipos de rutinas: los PROCEDIMIENTOS y las FUNCIONES.

Pero la utilización de subprogramas se considera de forma especial en el PASCAL, lenguaje enfocado a conseguir programas bien estructurados. Con las rutinas se dispone de una estructura muy apropiada para practicar el método de desarrollo en etapas sucesivas que consiste en abordar el diseño de los programas descomponiendo el problema en partes más sencillas y repitiendo este proceso sucesivamente hasta que la solución de cada una de las partes sea trivial. Entonces resulta fácil desarrollar cada uno de los elementos del programa, depurarlos y probarlos independientemente hasta que funcionen correctamente.

Con ese enfoque, el programa queda organizado en módulos que se expresan en forma de rutinas. Una recomendación muy extendida propone que la escritura de cada subprograma no sobrepase el tamaño de un folio para facilitar su lectura y comprensión.

En definitiva, el diseño modular va dirigido a conseguir programas fiables y legibles:

- El esfuerzo de la programación se concentra finalmente en subproblemas pequeños y sencillos
- Es fácil escribir módulos pequeños libres de error y verificarlos independientemente con datos simulados
- Los listados pequeños son más legibles
- Resulta natural la adaptación al trabajo en equipo. Las tareas de desarrollar los módulos se pueden repartir entre varias personas.

El interés de utilizar subprogramas también se justifica por la construcción de bibliotecas de rutinas de utilidad general que puedan ser invocadas desde los programas que escribe el usuario para resolver casos particulares.

En PASCAL, se ofrece además una posibilidad interesante: las rutinas pueden llamarse a sí mismas (RECURSIVIDAD); lo que permite plantear algoritmos simples para resolver algunos problemas cuya solución es difícil con los medios tradicionales de la SECUENCIA, las SENTENCIAS CONDICIONALES y la ITERACION.

## 7.1 CONCEPTOS

Una rutina es un conjunto de sentencias que tienen asociado un identificador y se ejecutan como un grupo cuando se le invoca desde la sección ejecutable del programa.

En las rutinas se pueden utilizar y modificar variables del resto del programa en determinadas condiciones.

Hay algunos procedimientos y funciones que están predefinidos por el compilador (READ, WRITE, EOLN, EOF, NEW, etc...); pero el usuario puede definir otros declarándolos de forma semejante a como se hace con otras entidades (constantes, tipos, variables).

Las FUNCIONES se distinguen de los procedimientos en que devuelven un valor único asociado a su identificador, del tipo correspondiente a la declaración.

Las rutinas tienen una estructura análoga a la del programa. Se componen de:

- . el ENCABEZAMIENTO
- . el CUERPO, con:
  - . Sección de declaraciones
  - . Sección ejecutable

En realidad, las rutinas son una especie de programas. Son SUBPROGRAMAS que admiten todo lo expuesto en los capítulos anteriores referente a estructuras de datos y de control; y que pueden incluir a otros subprogramas a su vez, configurando una estructura donde el PROGRAMA completo se contempla como un conjunto de MODULOS o BLOQUES anidados que pueden compartir e intercambiarse valores.

En este esquema, el programa principal y cada uno de los subprogramas constituyen ámbitos de acción diferentes donde se manejan entidades como constantes, etiquetas, tipos, variables, que deben haber sido declaradas previamente.

La parte del programa donde se tiene acceso a un identificador se llama ámbito o campo de existencia. Fuera de ese dominio, un identificador no tiene significado o tiene significado diferente. La regla general establece que **un identificador es conocido en el bloque donde ha sido declarado y en todos los que tiene anidados.**

### **Entidades globales, locales y estandar**

A las entidades que se declaran en el programa principal se les llama GLOBALES (variables globales, por ejemplo). Son conocidas a lo largo de todo el programa y existen durante toda la ejecución, si corresponden a estructuras estáticas.

A las entidades que sólo tienen significado en una parte del programa se les llama LOCALES. Son las que se declaran en un procedimiento o función y son conocidas sólo en el mismo y en cualquier otro que esté anidado dentro. Estos objetos sólo tienen la existencia asegurada mientras dura la ejecución del procedimiento donde han sido declaradas, después se libera el espacio que ocupaban en memoria quedando disponible para ser utilizado por otras.

En tercer lugar, se habla de objetos o entidades STANDARD, refiriéndose a los que están definidos en el lenguaje (compilador). Obviamente, éstos son conocidos en cualquier parte del programa.

En estas condiciones, queda claro que en un procedimiento se pueden manejar cualquier objeto GLOBAL y todos los que hayan sido declarados localmente; y que es necesario disponer de algún mecanismo para hacer posible la comunicación entre procedimientos diferentes. Esto se consigue mediante los PARAMETROS.

En lo que sigue, se van a tratar todos estos aspectos referentes al uso de las rutinas, considerando:

- . Cómo se declaran
- . Cómo funcionan los mecanismos para la comunicación entre módulos mediante parámetros
- . Cómo se invocan

```
PROGRAM Programa (INPUT, OUTPUT);
```

```
  CONST ...
```

```
  TYPE ...
```

```
  VAR ...
```

```
  PROCEDURE Rutina_1 ....
```

```
    CONST ...
```

```
    TYPE ...
```

```
    VAR ...
```

```
    PROCEDURE Rutina_2 ...
```

```
      CONST ...
```

```
      TYPE ...
```

```
      VAR ...
```

```
    BEGIN
```

```
      Sentencias ejecutables de Rutina_2
```

```
    END;
```

```
  BEGIN
```

```
    Sentencias ejecutables de Rutina_1
```

```
  END;
```

```
  PROCEDURE Rutina_3 ...
```

```
    CONST ...
```

```
    TYPE ...
```

```
    VAR ...
```

```
  BEGIN
```

```
    Sentencias ejecutables de Rutina_3
```

```
  END;
```

```
BEGIN
```

```
  Sentencias ejecutables de Programa
```

```
END.
```

## 7.2 LA DECLARACION

Como cualquier otra entidad definida por el usuario, los procedimientos (PROCEDURE) y las funciones (FUNCTION) deben declararse antes de ser utilizados. En la declaración se especifican todas las partes que los forman:

- . encabezamiento
- . declaraciones
- . sentencias ejecutables

En la sección de las declaraciones de un procedimiento o función, se pueden declarar otros subprogramas.

Sintaxis:

```

PROCEDURE identificador (lista de parámetros formales);
    Sección de las declaraciones;
BEGIN
    Sección ejecutable
END;

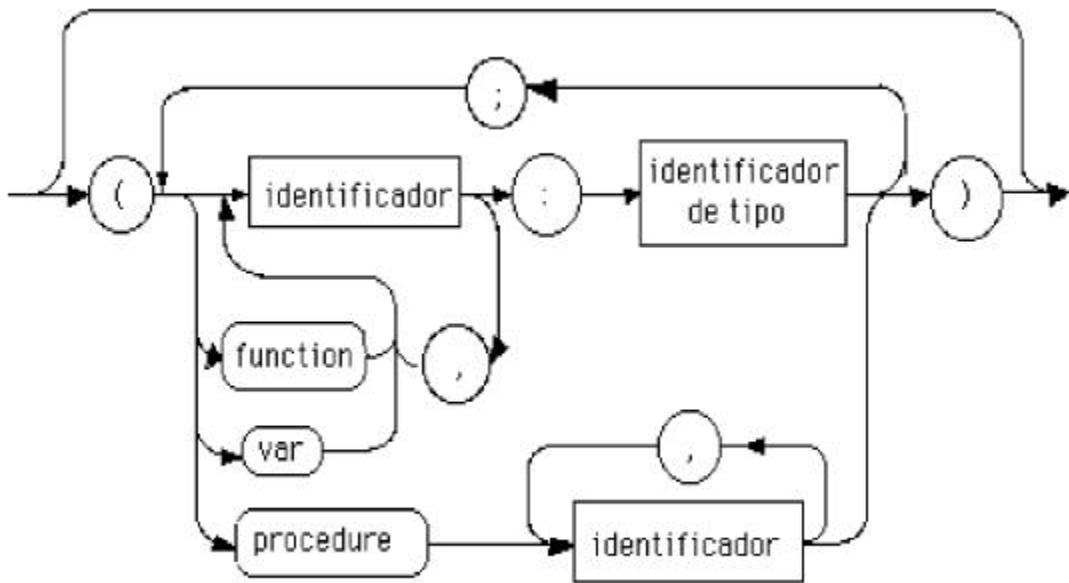
FUNCTION identificador (lista de parámetros formales) : Tipo ;
    Sección de las declaraciones;
BEGIN
    Sección ejecutable
END;
```

**Identificador:** da nombre a la rutina y se utiliza para invocarla al solicitar su ejecución. Se construye siguiendo las mismas reglas que rigen para las constantes y las variables.

**Lista de parámetros formales :** especificación de los identificadores que se utilizan para comunicar valores con otros módulos del programa. Se indican sus tipos y los mecanismos para la transferencia.

**Sección de las declaraciones :** contiene las declaraciones de todas las entidades definidas por el usuario que se utilizan dentro de la rutina: etiquetas, constantes, tipos, variables, procedimientos, funciones. Son identificadores locales.  
 No se admite la inicialización de variables.  
 No se puede volver a declarar los nombres de los parámetros formales.

**Sección ejecutable :** contiene las sentencias con las que se realizan las acciones que constituyen el subprograma. Termina con el delimitador END seguido de punto y coma.



### Lista de parámetros formales

En la declaración de una FUNCION es necesario especificar el tipo al que pertenece, lo que equivale a determinar los valores permitidos que puede llevar asociados el identificador de función. Sólo se admiten los tipos escalares y los punteros.

A continuación se muestran algunos ejemplos:

Ejemplo 1. Se leen caracteres por INPUT y se almacenan en una tabla. Incrementando un contador se conoce el número de caracteres leídos. En este caso no se han declarado parámetros. Se supone que "lon", "lonmax" y "cadena" son variables globales. No se han declarado objetos locales.

```
PROCEDURE Leer_cadena;
BEGIN
    lon:=0;
    WHILE (NOT EOLN) AND (lon<lonmax) DO
        BEGIN
            lon := lon+1;
            READ (cadena [lon] )
        END;
    READLN
END;
```

- Ejemplo 2. La función "Circulo" es de tipo REAL. Cuando se ejecute, se obtendrá un valor REAL asignado al identificador "Circulo". El parámetro formal es "radio"

```

FUNCTION Circulo ( radio: REAL ) : REAL;
  CONST  pi = 3.1416;
  BEGIN
    Circulo := pi * radio * radio
  END;

```

- Ejemplo 3. Aquí se leen valores por INPUT y se construye un conjunto con ellos. El tipo "dia" tiene que estar definido antes. La variable "dia\_libre" es local.

```

PROCEDURE Leer_conjunto ( VAR dias : SET OF dia);
  VAR  dia_libre : dia;
  BEGIN
    dias := [];
    WHILE NOT EOLN DO
      BEGIN
        READ (dia_libre);
        dias := dias + [dia_libre]
      END;
    READLN
  END;

```

- Ejemplo 4. Esta función busca al menor de los elementos de una tabla recorriendola entera. Los tipos "tablas" y "elemento" deben haber sido declarados antes. El tipo "tablas" tiene que ser ARRAY ...OF elemento. La variable "i" es una variable local.

```

FUNCTION Menor ( tabla : tablas , num : INTEGER) : elemento;
  VAR  i : INTEGER;
  BEGIN
    Menor := tabla [1];
    FOR i:=2 TO num DO IF tabla[i]<Menor THEN Menor:=tabla[i]
  END;

```

### 7.3 LOS PARAMETROS. MECANISMOS DE SUSTITUCION

En una rutina se puede manejar cualquier entidad GLOBAL.

Cuando el único objetivo que se persigue con la utilización de rutinas es segmentar el programa en módulos más pequeños para facilitar su legibilidad, no hay demasiados inconvenientes en manejar objetos de carácter global desde el interior de los subprogramas.

Pero el interés de utilizar rutinas se extiende y crece en importancia en los caos frecuentes donde hay que repetir la ejecución de grupos de sentencias a lo largo del programa, en contextos distintos y con valores iniciales diferentes. Entonces no conviene actuar sobre entidades globales desde el interior de los subprogramas para no correr el riesgo de modificar inadvertidamente valores externos al procedimiento o función. Es preferible actuar con entidades de ámbito local y disponer de alguna vía para relacionarlas con las definidas a nivel global y con las que son locales de otros subprogramas.

Hay ocasiones en las que la relación entre una rutina y el programa principal trasciende al caso de un programa determinado. Es cuando se escriben rutinas de utilidad general que se redactan de forma que cualquier usuario las pueda incluir en su programa.

En todos estos casos se necesita disponer de algún mecanismo para intercambiar valores entre las rutinas y el resto del programa. Ello se consigue mediante las listas de PARAMETROS, llamados también ARGUMENTOS.

Los parámetros que se indican en la declaración de las rutinas se llaman PARAMETROS FORMALES. Son identificadores de entidades locales con las que se pueden introducir y sacar valores en las rutinas. En las funciones, los parámetros formales sólo permiten la entrada, puesto que la salida produce un valor único que va asociado al identificador de la propia función.

Los parámetros que se indican en la llamada a las rutinas son los PARAMETROS ACTUALES.

Ejemplo:

```

...
PROCEDURE Altas ( codigo:articulos, cantidad:INTEGER,
                  fecha:fechas, seccion:CHAR);
...
Leer_datos;
Altas (datos.codigo, datos.cantidad, (12,3,1987), 'C');
...

```

En el ejemplo, "codigo", "cantidad", "fecha" y "seccion" son parámetros formales del procedimiento "Altas". Los parámetros actuales que se pasan en la llamada al procedimiento son "datos.codigo", "datos.cantidad", (12,3,1987), 'C'.

Cuando se ejecuta una llamada a rutina, se establece una asociación entre los parámetros actuales y los formales que permanece activa durante toda la ejecución del subprograma, haciendo posible el intercambio de valores con entidades locales de la rutina.

Ejemplo:

```

...
VAR      r, radio, circulo, esfera : REAL;
FUNCTION Potencia ( base:REAL, exponente:INTEGER):REAL;
  VAR    i : INTEGER;
  BEGIN
    Potencia := 1;
    FOR i:=1 TO exponente DO Potencia:=Potencia*base
  END;
...
circulo := 3.1416 * Potencia (radio, 2);
esfera := 4/3 * 3.1416 * Potencia (r, 3);

```

En el ejemplo, "base" y "exponente" son los argumentos de la función "Potencia". Son sus parámetros formales. La variable "i" es una variable local de "Potencia". Las variables "radio", "r" y las constantes "2" y "3" son los parámetros actuales que se pasan en las dos sentencias de llamada. Al ejecutarse la primera llamada para calcular la superficie de un círculo, se pasan los valores de "radio" y "2". Esos valores se asignan a los parámetros formales correspondientes, "base" y "exponente", y se realiza el cálculo devolviendo un valor de tipo REAL asignado al identificador "Potencia". Más adelante, se invoca a la misma función pasándole unos valores diferentes.

La correspondencia entre los parámetros formales y los actuales se establece por la posición que ocupan en las listas, y la sustitución se realiza aplicando algunos de los mecanismos siguientes: por VALOR, por REFERENCIA, por NOMBRE. Según el mecanismo que actúa, se habla de varias clases de parámetros:

### **Parámetros-valor**

En la sustitución por valor, se evalúa el parámetro actual y el valor resultante se asigna al parámetro formal correspondiente. El parámetro actual se puede expresar como una constante, una variable o una expresión.

El uso de un parámetro-valor consiste en la mera transferencia de un valor a la rutina. Estos son parámetros de entrada. Se pueden modificar dentro de la rutina, pero ello no afecta al parámetro actual correspondiente. No permiten sacar información de la rutina.

Los parámetros-valor se declaran indicando simplemente su nombre y su tipo, sin indicar prefijos. La ausencia de prefijos es precisamente lo que identifica a esta clase de parámetros.

Ejemplos:           FUNCTION Circulo (radio : REAL ) : REAL;  
                          ...  
                          FUNCTION Menor ( tabla: tablas, num : INTEGER):elemento;

### **Parámetros-variable**

Permiten aplicar el mecanismo de sustitución POR REFERENCIA, por el que se produce una asociación entre parámetro actual y parámetro formal que consiste en una especie de sustitución. En realidad se asignan al identificador de parámetro formal las mismas direcciones de memoria que corresponden al parámetro actual, de manera que cualquier modificación que se produzca dentro del procedimiento afecta a los parámetros actuales que son entidades externas al mismo.

Para describir gráficamente la situación, se puede imaginar una flecha para cada parámetro formal, que una su nombre con la dirección de memoria donde está almacenado su parámetro actual correspondiente. Toda operación que afecte al parámetro formal, en realidad se realiza sobre el parámetro actual.

Con este mecanismo de sustitución por referencia, se puede conseguir la transferencia de información con las rutinas en los dos sentidos: entrada y salida.

Los parámetros-variable se declaran precedidos de la palabra reservada VAR. Los parámetros actuales correspondientes tienen que ser variable del mismo tipo.

Con los ejemplos que se muestran a continuación, se pueden notar las diferencias entre los mecanismos de paso POR VALOR y POR REFERENCIA, por los efectos que producen.

```

PROGRAM P;
  VAR i : INTEGER;
      a : ARRAY [1..2] OF INTEGER;
  PROCEDURE Paso_por_valor ( x : INTEGER);
  BEGIN
      i := i+1; x := x+2
  END;
BEGIN
  a[1] := 10; a[2] := 20;
  i := 1;
  Paso_por_valor ( a[i] )
END.

```

Al ejecutarse el procedimiento, "x" es una variable local, de tipo INTEGER, que toma inicialmente el valor "10". Después se incrementa su valor en dos unidades, pero ello no afecta a la variable global "a". Después de que termine la ejecución del procedimiento, la variable "a" seguirá teniendo el mismo valor que antes ( a=(10,20)) y la variable "x" habrá dejado de existir.

```

PROGRAM P;
  VAR i : INTEGER;
      a : ARRAY [1..2] OF INTEGER;

  PROCEDURE Paso_por_referencia ( VAR x : INTEGER );
  BEGIN
      i := i+1; x := x+2
  END;
BEGIN
  a [1] := 10; a [2] := 20;
  i :=1 ;
  Paso_por_referencia ( a[i] )
END.

```

Aquí la sustitución se realiza sobre el primer elemento a[1]. La sentencia x:=x+2 significa realmente : a[1] := a[1]+2; de manera que después de ejecutarse el procedimiento, se habrá modificado el valor de la variable "a" ( a=(12,20)).

### Parámetros-procedimiento y parámetros-función

En la lista de parámetros formales, también se pueden incluir funciones y procedimientos. En tal caso hay que escribir su encabezamiento completo y tales rutinas sólo admiten parámetros pasados por valor.